Exercise 19

Abstract Data Type

For this exercise, you will be finishing the implementation of an abstract data type representing a sequence of integers. Internally, we will implement the sequence as a linked list, but we will use an incomplete type to hide all of this implementation from client code. That would let us change the implementation without breaking client code, as long as we provided equivalent functions for the client.

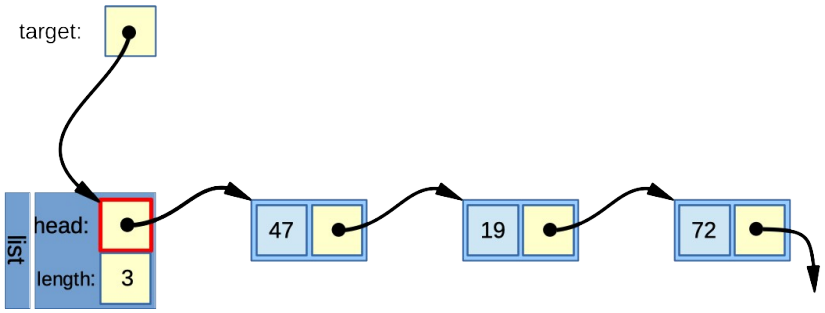
You'll find three source files, **list.h**, **list.c** and **listTest.c**, along with a test input and expected output file on the course Moodle page. You can download them using your web browser, or if you're working on remote.eos.ncsu.edu, you should be able to copy them to your current directory on using the following shell command:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise19/* .
```

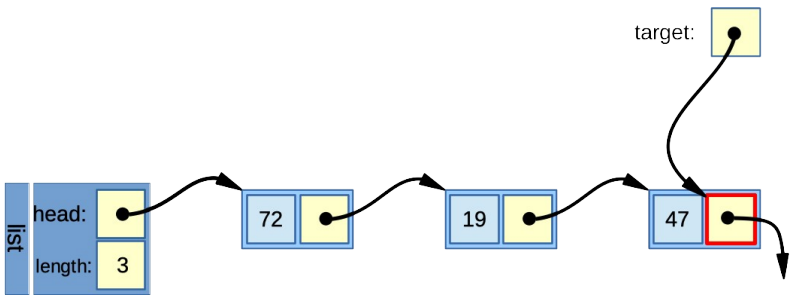
The **listTest.c** file contains the main() function. It reads a list of numbers and stores them in an instance of the List abstract data type created by **list.h** and **list.c**. In fact, it does this three times using different functions in list.c to add new values to the front of the list, to the back or in sorted order. You'll fill in the missing code in these functions. This will give you a good chance to practice using pointer-to-pointer to simplify your linked list code.

Here's what you'll need to do to fill in the missing parts of the list.c implementation:

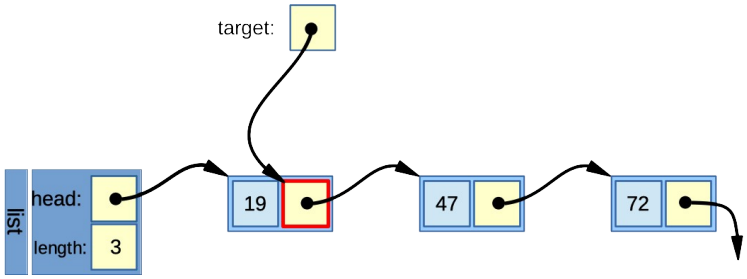
- The `insertFront()` function inserts the new value at the head of the list. We don't really need pointer-to-pointer to simplify this code, but it'll be a good warm-up, and we can re-use some of the code we figure out for this function in the later functions. As illustrated below, this function creates a pointer-to-pointer, **`target`** and sets it to point to the head pointer. Your job is to create a new node holding the given value, and add insert at the start of the list by using **`target`** to modify the head pointer.



- The `insertBack()` function inserts the new value at the tail of the list. Here, the **`target`** pointer is a little more useful. We're going to start it out pointing to the head pointer. Then, we're going to use **`target`** to walk down the list until it points to the null pointer marking the end of the list (even if that's the head pointer itself, like it will be when the list is empty). Then, just like with the previous function, we can use **`target`** modify the value of the pointer it points to and add a new node to the list.



- The `insertSorted()` function inserts the new value wherever it needs to go to keep the list in sorted order. This sounds like it might be tricky, but, really, it's almost the same as the `insertBack()` function. Again, we're going to start **`target`** out pointing to the head pointer. Then, we're going to use it to walk down the list until it points to either the null pointer at the end of the list or a pointer to a node containing a larger value than the one we're inserting. This will leave it pointing to the link we need to change to insert the new value. We can use **`target`** to modify this pointer and insert the new node at the right spot.



As usual, I've marked the places where you need to add code with ... in a comment. You'll need to add a line or two of code for each of these, instead of a whole block of code.

When you're done, you should be able to compile and link the test program with your list implementation using a command like the following:

```
$ gcc -Wall -std=c99 -g list.c listTest.c -o listTest
```

You should be able to run your program with input from the sample input file. Here, the first list has all the number from the input file in reverse order. The second has them in the same order as the file, and the last has them in sorted order.

```
$ ./listTest input.txt
12 elements : 29 51 75 15 88 12 36 8 31 47 19 72
12 elements : 72 19 47 31 8 36 12 88 15 75 51 29
12 elements : 8 12 15 19 29 31 36 47 51 72 75 88
```

If you want to make sure your program's output is exactly right, you can send the output to a file, then use diff to make sure it matches the expected output file.

When you're done, submit the source, **list.c**, to the exercise_19 assignment in Moodle. You shouldn't have needed to make any changes in the other two source files.